Mini AI Support Ticket System

Assignment Submission

✓ COMPLETED

Submission Deadline: 29 April 2026

GitHub Repository & Live Demo

Repository:

https://github.com/21mis7174/support_ticket_system

Live Demo:

Frontend: **http://13.233.109.44:3000**

Backend API: **http://13.233.109.44:8000**

API Documentation: **http://13.233.109.44:8000/docs**

Project Overview

A full-stack web application for managing support tickets with real-time updates, priority management, and status tracking. The system allows users to create tickets, view all tickets, and mark them as resolved.

Key Features:

- Create support tickets with title, description, and priority level

- View all tickets with sorting and filtering capabilities

- Filter tickets by status (All / Open / Resolved)

- Mark tickets as resolved with automatic timestamp tracking

- Real-time UI updates using React hooks

- Responsive design with Tailwind CSS

- Production-ready Docker deployment on AWS EC2

Technology Stack

Category	Technologies
Backend	Python 3.11, FastAPI, Pydantic, Motor (async MongoDB)
Frontend	Next.js 16, TypeScript, React 19, Tailwind CSS
Database	MongoDB 7.0 with Motor async driver
DevOps	Docker, Docker Compose, Docker BuildKit
Cloud	AWS EC2 (t2.micro), ap-south-1 region

Application Screenshots

Screenshot 1: Main Tickets List Page

The main dashboard showing all support tickets with status indicators, priority badges, and real-time filtering capabilities.

Figure 1: Tickets List Page - Shows all tickets with filters and resolve buttons

Screenshot 2: Create New Ticket Form

The form for creating new support tickets with title, description, and priority level selection.

Figure 2: Create New Ticket Form - Users can submit tickets with priority levels

Key UI Features Visible in Screenshots:

- ✓ Responsive ticket cards with title, description, and status

- ✓ Priority badges (Low, Medium, High) with color coding

- ✓ Status indicators (Open/Resolved) with visual differentiation

- ✓ Relative timestamp display ("2 minutes ago", "just now")

- ✓ One-click "Mark as Resolved" button

- ✓ Filter tabs for All/Open/Resolved tickets

- ✓ Create ticket form with dropdown for priority selection

- ✓ Loading states and smooth user interactions

API Endpoints

1. Create Ticket (POST /tickets)

Request:

```
curl -X POST http://13.233.109.44:8000/tickets \ -H "Content-Type: application/json" \ -d '{ "title": "Login page broken", "description": "Users cannot log in after deployment", "priority": "high" }'
```

Response (201 Created):

{ "id": "507f1f77bcf86cd799439011", "title": "Login page broken", "description": "Users cannot log in after deployment", "priority": "high", "status": "open", "created_at": "2026-04-27T18:09:46.484000Z" }

2. List All Tickets (GET /tickets)

Request:

```
curl http://13.233.109.44:8000/tickets | python3 -m json.tool
```

Response (200 OK): Returns array of all tickets with total count

3. Resolve Ticket (PATCH /tickets/{id}/resolve)

Request:

```
curl -X PATCH http://13.233.109.44:8000/tickets/507f1f77bcf86cd799439011/resolve
```

Response (200 OK): Returns updated ticket with resolved status and timestamp

Assignment Completion Checklist

PART 1 – Backend (FastAPI)

Requirement	Status
POST /tickets endpoint	✓ **Complete**
GET /tickets endpoint	✓ **Complete**
PATCH /tickets/{id}/resolve	✓ **Complete**
MongoDB Integration	✓ **Complete**
Pydantic Models & Validation	✓ **Complete**
Error Handling (422, 404, 400)	✓ **Complete**

PART 2 – Frontend (Next.js)

Requirement	Status
Create Ticket Page	✓ **Complete**
Tickets List Page	✓ **Complete**
Resolve Functionality	✓ **Complete**
Fetch API Integration	✓ **Complete**
Tailwind CSS Styling	✓ **Complete**
Loading & Error States	✓ **Complete**

PART 3 – Docker & Containerization

Requirement	Status
Backend Dockerfile (255MB optimized)	✓ **Complete**
Frontend Dockerfile (267MB optimized)	✓ **Complete**
docker-compose.yml orchestration	✓ **Complete**
Health checks for all services	✓ **Complete**
Non-root user execution	✓ **Complete**
BuildKit optimization	✓ **Complete**

PART 4 – Cloud Deployment

Requirement	Status
AWS EC2 deployment	✓ **Complete**
Public IP (13.233.109.44)	✓ **Complete**
MongoDB with 10 seeded tickets	✓ **Complete**
CORS configuration	✓ **Complete**
Auto-restart on failure	✓ **Complete**
UTC timezone handling	✓ **Complete**

PART 5 – Submission Requirements

Requirement	Status
GitHub repository link	✓ **Complete**
README with setup instructions	✓ **Complete**
API endpoints documentation	✓ **Complete**
Screenshots (home_page.png, new_ticket.png)	✓ **Complete**
Live demo link (http://13.233.109.44:3000)	✓ **Complete**

Quick Start Guide

Option A: Docker Compose (Recommended)

```
git clone https://github.com/21mis7174/support_ticket_system.git cd support_ticket_system
docker compose up -d # Access: # Frontend: http://localhost:3000 # Backend:
http://localhost:8000 # API Docs: http://localhost:8000/docs
```

Option B: Local Development

```
# Backend cd backend/ python3 -m venv .venv source .venv/bin/activate pip install -r requirements.txt python run.py # Frontend (new terminal) cd frontend/ npm install npm run dev
```

AWS EC2 Deployment Details

Instance Configuration:

- Instance Type: t2.micro (Free tier eligible)

- Region: ap-south-1 (Mumbai)

- AMI: Amazon Linux 2023

- Public IP: 13.233.109.44

Running Services:

- MongoDB 7.0 (Port 27017)

- FastAPI Backend (Port 8000)

- Next.js Frontend (Port 3000)

Testing the Application

You can test the live deployment at: **http://13.233.109.44:3000**

What to test:

- ✓ Create a new ticket and verify it appears in the list

- ✓ View all 10 seeded test tickets

- ✓ Click "Mark as Resolved" button and see status update

- ✓ Use status filters (All / Open / Resolved)

- ✓ Verify relative timestamps display correctly

- ✓ Test API endpoints via Swagger: **http://13.233.109.44:8000/docs**

Assignment Submission Summary

Position: Full Stack Developer | Mindtide.ai

Assignment: Mini AI Support Ticket System

Submitted: 27 April 2026

Deadline: 29 April 2026

GitHub Repository:

https://github.com/21mis7174/support_ticket_system

Live Demo:

http://13.233.109.44:3000

✓ All 5 parts of assignment completed

✓ Production-ready code with Docker

✓ Live deployment on AWS EC2

✓ Complete API documentation

✓ Screenshots included

✓ Comprehensive README and setup instructions